

Reviewer Pack — Minimal Number of Automorphic Forms and Communication Comple...

Entry 9a81a2df52a8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 08:08:29 UTC

Minimal Number of Automorphic Forms and Communication Complexity Rank Variance

Entry ID: 9a81a2df52a8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 08:08:29 UTC

1. Conjecture

Field A (mathematical branch): Automorphic Representation Theory **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

For any given boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the number of distinct automorphic forms associated with the matrix representation of its communication complexity is linearly correlated with its rank variance, i.e., $|\text{Aut}(F)| = \Theta(\rho(f))$, where $\rho(f)$ denotes the rank variance of f .

Rationale (proposer's reasoning):

Automorphic representation theory studies the symmetries in number theory, and automorphic forms are functions that transform under these symmetries. If this conjecture holds, it would imply a deep connection between the symmetry properties of these functions and their complexity as communication tasks. This could potentially lead to new insights into both areas.

Taxonomy category: AUTOMORPHIC_REPRESENTATION_THEORY × COMMUNICATION_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 66c0d1529f5b29fa

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the number of distinct automorphic forms ($|\text{Aut}(F)|$) and the rank variance ($\rho(f)$) of the communication complexity matrices for all $n \leq 40$ boolean functions f is ≥ 0.8 with $p\text{-value} \leq 0.05$, calculated over at least 30 independent function seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 10 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - automorphic representation theory AND communication complexity matrix rank variance - rank variance automorphic forms communication complexity - matrix representation boolean function automorphic form rank variance

Top relevant hits considered: - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1005.0979v1] Supersymmetry in Random Matrix Theory - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/1906.03766v3] Variance Reduction in Gradient Exploration for On-line Learning to Rank - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.randint(0, 1) for _ in range(2**n)]

    def matrix_representation(f, n):
        M = []
        for x in range(2**n):
            row = []
            for y in range(2**n):
                row.append(int(f[x] == f[y]))
            M.append(row)
        return M
```

```

def gaussian_elimination(M):
    n = len(M)
    A = [row[:] for row in M]
    lead = 0
    for r in range(n):
        if lead >= n:
            break
        i = r
        while A[i][lead] == 0:
            i += 1
            if i == n:
                i = r
                lead += 1
            if lead == n:
                return A, False
        A[r], A[i] = A[i], A[r]
        for i in range(r + 1, n):
            factor = Fraction(A[i][lead], A[r][lead])
            for j in range(lead, n):
                A[i][j] -= factor * A[r][j]
    return A, True

def rank_variance(M):
    A, success = gaussian_elimination(M)
    if not success:
        return 0
    rank = sum(1 for row in A if any(row))
    return (rank - n) ** 2 / n

def count_automorphic_forms(M):
    n = len(M)
    forms = set()
    for i in range(n):
        for j in range(i + 1, n):
            if all(M[i][k] == M[j][k] for k in range(n)):
                forms.add(tuple(M[i]))
    return len(forms)

def communication_complexity(f, n):
    M = matrix_representation(f, n)
    return rank_variance(M)

def correlation_coefficient(x, y):
    mean_x = sum(x) / len(x)
    mean_y = sum(y) / len(y)
    cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(len(x))) / len(x)
    std_x = math.sqrt(sum((x[i] - mean_x) ** 2 for i in range(len(x))) / len(x))
    std_y = math.sqrt(sum((y[i] - mean_y) ** 2 for i in range(len(y))) / len(y))
    return cov / (std_x * std_y)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    f = generate_boolean_function(n)
    rho_f = communication_complexity(f, n)
    aut_forms = count_automorphic_forms(M)

```

```

        results.append((rho_f, aut_forms))

    if len(results) < 30:
        return {
            "metric_name": "Correlation Coefficient",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "Insufficient instances"
        }

    rho_values, aut_form_counts = zip(*results)
    corr_coeff = correlation_coefficient(rho_values, aut_form_counts)
    support_fraction = sum(1 for cc in corr_coeff if cc >= 0.8) / len(corr_coeff)

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": corr_coeff,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_corr_coeff = sum(result["metric_value"] for result in results) / len(results)
        std_corr_coeff = math.sqrt(sum((result["metric_value"] - mean_corr_coeff) ** 2 for result
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_corr_coeff} std={std_corr_coeff} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e8e972f8.py", line 119, in <module>

```
trial_result = run_trial(seed)
^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e8e972f8.py", line 87, in run_trial
    aut_forms = count_automorphic_forms(M)
                                   ^
```

NameError: name 'M' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation coefficient and p-value as required by the support condition. | next: Investigate the cause of the crash in the test code and run the test again to ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14157
2	preregistration	ollama_remote	glm4:latest	0	0	10816
3	novelty	ollama_remote	glm4:latest	0	0	8316
4	novelty	ollama_remote	glm4:latest	0	0	10982
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31705
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13523
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13265
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13354
9	judge	ollama_remote	glm4:latest	0	0	16151

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132268 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9a81a2df52a8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9a81a2df52a8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9a81a2df52a8.tar.gz (if generated)